Data Quality Standards

> **Document Owner:** Platform Architecture Team | **Status:** Approved | **Last Updated:** February 2026

Overview

This document establishes enterprise-wide data quality standards using Databricks native capabilities. Data quality is enforced at three levels:

Level	Technology	Use Case
Pipeline	DLT Expectations	Real-time validation in streaming/batch pipelines
Classic Jobs	DQX Library	Quality checks in non-DLT Spark jobs
Monitoring	Lakehouse Monitor	Time-series trends, drift detection, profiling

Golden Rules

ID	Rule	Severity	WAF Pillar
DQ-01	All DLT pipelines must have data quality expectations	Critical	Reliability
DQ-02	Classic Spark jobs must use DQX for quality validation	High	Reliability
DQ-03	Gold layer tables must have Lakehouse Monitors	Critical	Reliability
DQ-04	Quality failures must be captured, not silently dropped	Critical	Reliability
DQ-05	Enable schema-level anomaly detection for freshness and completeness	Critical	Reliability
DQ-06	Configure data profiling with baseline tables for drift detection	Required	Reliability
DQ-07	Define custom metrics for business KPIs in Lakehouse Monitors	Required	Reliability

DQ-01: DLT Expectations (Declarative Pipelines)

Delta Live Tables (DLT) provides built-in data quality expectations for real-time validation during pipeline execution.

Expectation Types

Decorator	Behavior	Use For
`@dlt.expect`	Logs warning, keeps row	Soft validation, monitoring
`@dlt.expect_or_drop`	Drops invalid rows silently	Non-critical filters
`@dlt.expect_or_fail`	Fails pipeline	Critical data integrity
`@dlt.expect_all`	Multiple expectations, log	Combined soft checks
`@dlt.expect_all_or_drop`	Multiple expectations, drop	Combined filters
`@dlt.expect_all_or_fail`	Multiple expectations, fail	Combined critical checks

Standard Expectations by Layer

Silver Layer (Cleansing):

```
import dlt
from pyspark.sql.functions import col

@dlt.table(
    name="silver_orders",
    comment="Cleansed orders with data quality expectations"
)
@dlt.expect("valid_order_id", "order_id IS NOT NULL")
@dlt.expect("valid_amount", "order_amount >= 0")
@dlt.expect_or_drop("valid_date", "order_date IS NOT NULL")
@dlt.expect_or_fail("valid_customer", "customer_id IS NOT NULL")
def silver_orders():
    return dlt.read_stream("bronze_orders")
```

Gold Layer (Business Rules):

```
@dlt.table(name="gold_daily_sales")
@dlt.expect_all_or_fail({
    "valid_revenue": "total_revenue >= 0",
    "valid_units": "units_sold >= 0",
    "valid_date": "sales_date IS NOT NULL"
})
def gold_daily_sales():
    return dlt.read("silver_orders").groupBy("sales_date").agg(...)
```

Quarantine Pattern

Capture failed records instead of dropping them:

```
# Main table - valid records only
@dlt.table(name="silver_orders")
```

```python
@dlt.expect_all_or_drop({
    "valid_order_id": "order_id IS NOT NULL",
    "valid_amount": "order_amount >= 0"
})
def silver_orders():
    return dlt.read_stream("bronze_orders")

# Quarantine table - invalid records for review
@dlt.table(
    name="silver_orders_quarantine",
    comment="Invalid orders for data steward review"
)
def silver_orders_quarantine():
    return (
        dlt.read_stream("bronze_orders")
        .filter("order_id IS NULL OR order_amount < 0")
        .withColumn("quarantine_reason",
            when(col("order_id").isNull(), "NULL_ORDER_ID")
            .when(col("order_amount") < 0, "NEGATIVE_AMOUNT")
            .otherwise("UNKNOWN"))
        .withColumn("quarantine_timestamp", current_timestamp())
    )
```

Viewing Expectation Results

```sql
-- Query DLT event log for expectation results
SELECT
    timestamp,
    details:flow_name AS pipeline_name,
    details:expectation_name AS expectation,
    details:passed_records AS passed,
    details:failed_records AS failed
FROM event_log(TABLE(my_catalog.my_schema.__event_log))
WHERE event_type = 'flow_progress'
  AND details:expectation_name IS NOT NULL
ORDER BY timestamp DESC;
```

DQ-02: DQX Library (Classic Spark Jobs)

For non-DLT Spark jobs (Bronze setup, Gold merge), use **DQX** (Databricks Labs Data Quality eXtensions) for validation with detailed diagnostics.

Installation

```bash
%pip install databricks-labs-dqx
```

DQX vs DLT Expectations

Feature	DLT Expectations	DQX
Environment	DLT pipelines only	Any Spark job
Configuration	Python decorators	YAML or Python
Failure detail	Pass/fail counts	Row-level diagnostics
Quarantine	Manual pattern	Built-in split
Custom checks	Limited	Extensible

Basic Usage

```
from databricks.labs.dqx.engine import DQEngine
from databricks.labs.dqx.col_functions import is_not_null,
is_not_less_than

# Initialize engine
dq_engine = DQEngine(spark)

# Define checks
checks = [
    is_not_null("order_id"),
    is_not_null("customer_id"),
    is_not_less_than("order_amount", limit=0),
]

# Apply checks and split valid/invalid
valid_df, invalid_df = dq_engine.apply_checks_by_metadata_and_split(
    df=source_df,
    checks=checks
)

# Write valid records to Gold
valid_df.write.format("delta").mode("append").saveAsTable("gold.orders")

# Write invalid records to quarantine
invalid_df.write.format("delta").mode("append").saveAsTable("gold.orders_q
uarantine")
```

YAML Configuration

Store rules in YAML for maintainability:

```
# quality_rules/orders.yaml
checks:
  - name: valid_order_id
    check: is_not_null
```

```yaml
    column: order_id
    criticality: error

  - name: valid_amount
    check: is_not_less_than
    column: order_amount
    params:
      limit: 0
    criticality: error

  - name: valid_customer
    check: is_not_null
    column: customer_id
    criticality: error
```

```python
from databricks.labs.dqx.engine import DQEngine

dq_engine = DQEngine(spark)

# Load checks from YAML
valid_df, invalid_df = dq_engine.apply_checks_by_metadata_and_split(
    df=source_df,
    checks_file="quality_rules/orders.yaml"
)
```

Gold Layer Pre-Merge Validation

```python
def merge_with_quality_checks(spark, silver_df, gold_table):
    """Merge Silver to Gold with DQX validation."""

    from databricks.labs.dqx.engine import DQEngine
    from databricks.labs.dqx.col_functions import is_not_null,
is_not_less_than

    dq_engine = DQEngine(spark)

    # Define quality checks
    checks = [
        is_not_null("customer_key"),
        is_not_null("order_date"),
        is_not_less_than("order_amount", limit=0),
    ]

    # Split valid/invalid
    valid_df, invalid_df = dq_engine.apply_checks_by_metadata_and_split(
        df=silver_df,
        checks=checks
    )
```

```
    # Log quality metrics
    total = silver_df.count()
    valid_count = valid_df.count()
    invalid_count = invalid_df.count()
    print(f"Quality: {valid_count}/{total} passed
({100*valid_count/total:.1f}%)")

    # Quarantine invalid records
    if invalid_count > 0:
        invalid_df.write.format("delta").mode("append").saveAsTable(f"
{gold_table}_quarantine")
        print(f"Quarantined {invalid_count} records")

    # Merge valid records
    from delta.tables import DeltaTable
    delta_gold = DeltaTable.forName(spark, gold_table)
    delta_gold.alias("target").merge(
        valid_df.alias("source"),
        "target.customer_key = source.customer_key"
    ).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

    return valid_count, invalid_count
```

Available DQX Check Functions

Function	Description	Example
`is_not_null`	Column is not null	`is_not_null("id")`
`is_not_less_than`	Value >= limit	`is_not_less_than("amount", limit=0)`
`is_not_greater_than`	Value <= limit	`is_not_greater_than("age", limit=150)`
`is_in_list`	Value in allowed set	`is_in_list("status", ["A", "B", "C"])`
`matches_regex`	Matches pattern	`matches_regex("email", r".*@.*\\..*")`
`is_unique`	No duplicates	`is_unique("order_id")`

DQ-03: Lakehouse Monitoring (Time-Series Trends)

Lakehouse Monitor provides automated profiling, drift detection, and anomaly detection for Gold layer tables.

When to Use Lakehouse Monitoring

Capability	Description
Data Profiling	Summary statistics, null rates, distributions
Drift Detection	Compare current data to baseline or previous window
Anomaly Detection	Detect freshness and completeness issues

Capability	Description
Custom Metrics	Business KPIs tracked over time

Creating a Monitor

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.catalog import MonitorTimeSeries

w = WorkspaceClient()

# Create time-series monitor
monitor = w.quality_monitors.create(
    table_name="catalog.gold.fact_daily_sales",
    assets_dir="/Shared/monitors/fact_daily_sales",
    output_schema_name="catalog.gold_monitoring",
    time_series=MonitorTimeSeries(
        timestamp_col="sales_date",
        granularities=["1 day"]
    ),
    schedule=MonitorCronSchedule(
        quartz_cron_expression="0 0 8 * * ?",  # Daily at 8 AM
        timezone_id="America/Los_Angeles"
    ),
    custom_metrics=[
        MonitorMetric(
            type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
            name="daily_revenue",
            input_columns=["total_revenue"],
            definition="sum(total_revenue)",
            output_data_type="DOUBLE"
        ),
        MonitorMetric(
            type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
            name="null_customer_rate",
            input_columns=["customer_id"],
            definition="sum(case when customer_id is null then 1 else 0
end) / count(*)",
            output_data_type="DOUBLE"
        )
    ]
)
```

Monitor Output Tables

Each monitor creates three output tables:

Table	Content
`{table}_profile_metrics`	Column statistics per time window

Table	Content
`{table}_drift_metrics`	Drift scores vs baseline or previous window
`{table}_analysis`	Anomaly detection results

Querying Monitor Results

Check null rate trends:

```sql
SELECT
    window_start,
    column_name,
    percent_null
FROM catalog.gold_monitoring.fact_daily_sales_profile_metrics
WHERE column_name = 'customer_id'
ORDER BY window_start DESC
LIMIT 30;
```

Check drift scores:

```sql
SELECT
    window_start,
    column_name,
    drift_type,
    statistic,
    drift_score
FROM catalog.gold_monitoring.fact_daily_sales_drift_metrics
WHERE drift_score > 0.1  -- Significant drift
ORDER BY window_start DESC;
```

Check anomalies:

```sql
SELECT
    detection_time,
    anomaly_type,
    severity,
    description
FROM catalog.gold_monitoring.fact_daily_sales_analysis
WHERE severity IN ('HIGH', 'CRITICAL')
ORDER BY detection_time DESC;
```

Standard Monitors by Table Type

Fact Tables:

- Timestamp column: Transaction date

- Granularity: 1 day
- Custom metrics: Revenue totals, transaction counts, null rates

Dimension Tables (SCD2):

- Timestamp column: `effective_from`
- Granularity: 1 day
- Custom metrics: Active record counts, version distributions

Alerting on Monitor Results

```sql
-- Create alert for high null rates
SELECT
    window_start,
    column_name,
    percent_null
FROM catalog.gold_monitoring.fact_daily_sales_profile_metrics
WHERE column_name = 'customer_id'
  AND percent_null > 0.05  -- Alert if >5% nulls
  AND window_start >= current_date() - INTERVAL 1 DAY;
```

DQ-04: Failure Capture Requirements

Never silently drop invalid data. Always capture failures for analysis.

Quarantine Table Pattern

Every table with quality rules must have a corresponding quarantine table:

Main Table	Quarantine Table
`gold.dim_customer`	`gold.dim_customer_quarantine`
`gold.fact_orders`	`gold.fact_orders_quarantine`

Quarantine Table Schema

```sql
CREATE TABLE gold.fact_orders_quarantine (
    -- Original columns
    order_id STRING,
    customer_id STRING,
    order_amount DECIMAL(18,2),
    order_date DATE,

    -- Quality metadata
    quarantine_reason STRING NOT NULL,
    failed_checks ARRAY<STRING>,
    quarantine_timestamp TIMESTAMP NOT NULL,
    source_file STRING
```

```
)
USING DELTA
COMMENT 'Quarantined orders that failed quality checks for data steward
review.';
```

Quarantine Review Process

1. **Daily Review:** Data stewards review quarantine tables
2. **Root Cause:** Identify source system issues
3. **Remediation:** Fix upstream or adjust rules
4. **Reprocessing:** Move corrected records to main table

DQ-05: Schema-Level Anomaly Detection

Enable anomaly detection at the schema level to automatically monitor **freshness** (commit cadence) and **completeness** (row volume) across all tables without per-table configuration.

Why It Matters

- Detects stale tables before downstream consumers are impacted
- Identifies unexpected row volume drops (partial loads, failed ingestion)
- Zero per-table configuration — automatically aligns scan frequency with table update cadence
- Does not modify monitored tables or add overhead to jobs

Requirements

Requirement	Details
Unity Catalog	Must be enabled
Serverless compute	Must be enabled
Privileges	`MANAGE SCHEMA` or `MANAGE CATALOG`

How It Works

Check	What It Does	Alert Condition
Freshness	Analyzes commit history to predict expected update timing	Table commits are unusually delayed vs historical pattern
Completeness	Predicts expected 24-hour row ranges from historical trends	Row count falls below predicted threshold
Percent Null (Beta)	Monitors null percentages per column	Null rate exceeds historical norm

Enabling Anomaly Detection

Anomaly detection is enabled at the schema level via Catalog Explorer or the Governance Hub:

1. Navigate to **Catalog Explorer** → select schema
2. Enable **Data Quality Monitoring** for the schema
3. The system automatically begins scanning all tables in the schema

Excluding Tables

Use the API to exclude specific tables (staging, temporary, or internal tables):

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

# Exclude tables that don't need anomaly monitoring
w.quality_monitors.update(
    table_name="catalog.gold",
    excluded_table_full_names=[
        "catalog.gold.stg_temp_orders",
        "catalog.gold.debug_log"
    ]
)
```

Alerting on Anomalies

Query the system table to create SQL alerts:

```
-- Alert on freshness or completeness anomalies
SELECT
    table_name,
    check_type,
    status,
    detection_date,
    reason,
    downstream_impact
FROM system.data_quality_monitoring.table_results
WHERE status IN ('STALE', 'INCOMPLETE')
  AND detection_date >= current_date() - INTERVAL 1 DAY
ORDER BY detection_date DESC;
```

Anti-Patterns

- Monitoring individual tables manually instead of enabling schema-level detection
- Ignoring freshness anomalies because "the job just ran late"
- Not setting up alerts on `system.data_quality_monitoring.table_results`

DQ-06: Data Profiling with Baseline Tables

Configure data profiling on all Gold layer tables with appropriate profile types and baseline tables to enable meaningful drift detection.

Why It Matters

- Profile metrics reveal null rates, distributions, and statistical changes over time
- Baseline tables establish the "expected" data distribution for drift comparison
- Drift detection catches subtle data quality degradation that row-level checks miss
- Supports both data pipelines (time series) and ML workflows (inference)

Profile Types

Type	Use For	Timestamp	Baseline
Time Series	Fact tables, event data	Transaction/event date	Historical window of known-good data
Inference	ML model request logs	Request timestamp	Training/validation dataset
Snapshot	Dimension tables, reference data	None (full table scan)	Previous snapshot

Choosing the Right Profile Type

```
What type of table?
├── Has timestamp + regular updates → Time Series
│   └── Is an ML inference table? → Inference
├── Slowly changing dimension → Snapshot
└── Reference/lookup table → Snapshot
```

Creating a Profiled Monitor with Baseline

```python
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.catalog import MonitorTimeSeries

w = WorkspaceClient()

monitor = w.quality_monitors.create(
    table_name="catalog.gold.fact_daily_sales",
    assets_dir="/Shared/monitors/fact_daily_sales",
    output_schema_name="catalog.gold_monitoring",
    time_series=MonitorTimeSeries(
        timestamp_col="sales_date",
        granularities=["1 day"]
    ),
    # Baseline table for drift comparison
    baseline_table_name="catalog.gold.fact_daily_sales_baseline"
)
```

Profile Output Tables

Each profiled monitor generates two Delta tables:

Output Table	Content	Use
`{table}_profile_metrics`	Per-column statistics per time window: null rate, distinct count, percentiles, distributions	Trend analysis, SLA monitoring
`{table}_drift_metrics`	Drift scores comparing current window to baseline or previous window	Detecting distribution shifts

Querying Profile Metrics

```sql
-- Track null rate trends for critical columns
SELECT
    window_start,
    column_name,
    percent_null,
    num_nulls,
    distinct_count,
    avg,
    stddev
FROM catalog.gold_monitoring.fact_daily_sales_profile_metrics
WHERE column_name IN ('customer_id', 'order_amount', 'product_id')
  AND window_start >= current_date() - INTERVAL 30 DAY
ORDER BY window_start DESC, column_name;
```

Querying Drift Metrics

```sql
-- Detect significant distribution drift
SELECT
    window_start,
    column_name,
    drift_type,
    statistic,
    drift_score
FROM catalog.gold_monitoring.fact_daily_sales_drift_metrics
WHERE drift_score > 0.1
ORDER BY drift_score DESC;
```

Anti-Patterns

- Using snapshot profiling for time-series data (loses temporal context)
- Not setting a baseline table (drift is computed against previous window only — misses gradual degradation)
- Profiling only after incidents instead of continuously

- Ignoring drift in dimension tables (can cause silent join failures)

DQ-07: Custom Metrics for Business KPIs

Define custom metrics in Lakehouse Monitors to track business-specific KPIs beyond standard profiling statistics.

Why It Matters

- Standard profiling covers statistical measures but not business rules
- Custom metrics track domain-specific KPIs (revenue totals, conversion rates, SLA compliance)
- Derived metrics minimize recomputation by building on aggregate results
- Drift metrics detect business metric changes across time windows

Three Metric Types

Type	Computes From	Stored In	Use For
Aggregate	Primary table columns directly	Profile metrics table	Business KPIs: revenue, counts, rates
Derived	Previously calculated aggregate metrics	Profile metrics table	Computed measures: ratios, scores
Drift	Aggregate/derived metrics across time windows	Drift metrics table	Change detection: week-over-week, vs baseline

Aggregate Metrics

Calculate directly from table columns using SQL expressions:

```
from databricks.sdk.service.catalog import MonitorMetric,
MonitorMetricType

# Revenue KPI
revenue_metric = MonitorMetric(
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
    name="total_revenue",
    input_columns=["total_amount"],
    definition="sum(total_amount)",
    output_data_type="DOUBLE"
)

# Null rate for a critical column
null_rate_metric = MonitorMetric(
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
    name="customer_null_rate",
    input_columns=["customer_id"],
    definition="sum(case when customer_id is null then 1 else 0 end) /
count(*)",
```

```
    output_data_type="DOUBLE"
)

# Multi-column metric using :table indicator
avg_order_value = MonitorMetric(
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
    name="avg_order_value",
    input_columns=[":table"],
    definition="sum(total_amount) / count(distinct order_id)",
    output_data_type="DOUBLE"
)
```

Derived Metrics

Build on aggregate metrics to avoid recomputation:

```
# Derived from aggregate metrics (no primary table access)
revenue_per_customer = MonitorMetric(
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_DERIVED,
    name="revenue_per_customer",
    input_columns=["total_revenue", "distinct_customer_count"],
    definition="total_revenue / nullif(distinct_customer_count, 0)",
    output_data_type="DOUBLE"
)
```

Drift Metrics

Compare metrics across time windows using Jinja templates:

```
# Week-over-week revenue change
revenue_drift = MonitorMetric(
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_DRIFT,
    name="revenue_wow_change",
    input_columns=["total_revenue"],
    definition="(({{current_df}}.total_revenue -
{{base_df}}.total_revenue) "
                "/ nullif({{base_df}}.total_revenue, 0))",
    output_data_type="DOUBLE"
)
```

Standard Custom Metrics by Table Type

Fact Tables:

Metric	Type	Definition
`total_revenue`	Aggregate	`sum(total_amount)`

Metric	Type	Definition
`transaction_count`	Aggregate	`count(*)`
`avg_order_value`	Derived	`total_revenue / transaction_count`
`null_key_rate`	Aggregate	`sum(case when pk_col is null then 1 else 0 end) / count(*)`
`revenue_wow_change`	Drift	Week-over-week revenue comparison

Dimension Tables:

Metric	Type	Definition
`active_record_count`	Aggregate	`sum(case when is_current = true then 1 else 0 end)`
`version_count`	Aggregate	`count(distinct customer_key)`
`completeness_score`	Aggregate	Multi-column null check via `:table`

Constraints

- Custom metric definitions must be **single SQL expressions** — no joins or subqueries
- Use `input_columns=[":table"]` when the calculation involves multiple columns
- Use Jinja templates (`{{input_column}}`, `{{current_df}}`, `{{base_df}}`) for parameterized definitions
- Output data type must be specified as a Spark datatype JSON string

Anti-Patterns

- Defining all metrics as aggregate when derived would avoid redundant computation
- Not tracking drift metrics (only seeing current values, missing trends)
- Using complex subqueries in definitions (will fail — use derived metrics instead)
- Omitting `input_columns=[":table"]` for multi-column calculations (metric won't compute correctly)

Validation Checklist

DLT Pipelines

- ☐ All Silver tables have `@dlt.expect` or `@dlt.expect_or_drop` decorators
- ☐ Critical validations use `@dlt.expect_or_fail`
- ☐ Quarantine tables capture dropped records
- ☐ Expectation results are monitored via event log

Classic Spark Jobs

- ☐ DQX library installed in job environment
- ☐ Quality checks defined in YAML or Python

- [ ] `apply_checks_by_metadata_and_split` used for valid/invalid separation
- [ ] Invalid records written to quarantine table
- [ ] Quality metrics logged

Lakehouse Monitoring

- [ ] All Gold fact tables have monitors
- [ ] Monitors have appropriate timestamp columns
- [ ] Custom metrics defined for business KPIs (DQ-07)
- [ ] Drift detection enabled with baseline tables (DQ-06)
- [ ] Alerts configured for anomalies

Anomaly Detection (DQ-05)

- [ ] Schema-level anomaly detection enabled for Gold schemas
- [ ] `system.data_quality_monitoring.table_results` alerts configured
- [ ] Staging/temporary tables excluded from monitoring
- [ ] Freshness and completeness checks active

Data Profiling (DQ-06)

- [ ] Correct profile type selected (time series / inference / snapshot)
- [ ] Baseline table configured for drift comparison
- [ ] Profile metrics queried for null rate and distribution trends
- [ ] Drift metrics reviewed for significant shifts (score > 0.1)

Custom Metrics (DQ-07)

- [ ] Aggregate metrics defined for business KPIs (revenue, counts, rates)
- [ ] Derived metrics used where possible to avoid recomputation
- [ ] Drift metrics configured for week-over-week change detection
- [ ] Multi-column metrics use `input_columns=[":table"]`

Quarantine Management

- [ ] Every validated table has a quarantine table
- [ ] Quarantine includes failure reason
- [ ] Quarantine includes timestamp
- [ ] Review process documented
- [ ] Retention policy defined

Quick Reference

Expectation Decision Tree

```
Is this a DLT pipeline?
├── Yes → Use @dlt.expect decorators
│   ├── Non-critical → @dlt.expect (log only)
```

```
│   ├── Filter bad data → @dlt.expect_or_drop + quarantine
│   └── Critical → @dlt.expect_or_fail
└── No → Use DQX library
    ├── Define checks (YAML or Python)
    ├── Split valid/invalid
    └── Write invalid to quarantine
```

Monitor Decision Tree

```
What type of table?
├── Gold Fact Table → Time-series monitor on transaction date
├── Gold Dimension (SCD2) → Time-series on effective_from
├── Inference Table → Time-series on request timestamp
└── Aggregate Table → Snapshot monitor (no timestamp)
```

Quality Metrics to Track

Metric	Threshold	Action
Null rate	< 1% (critical cols)	Alert data steward
Duplicate rate	0% (PKs)	Fail pipeline
Out-of-range	< 0.1%	Review business rules
Drift score	< 0.1	Investigate root cause
Freshness	Within SLA	Page on-call

Related Documents

- Data Governance
- Data Modeling
- Naming & Comment Standards
- Silver Layer Patterns
- Gold Layer Patterns
- Lakehouse Monitoring

References

- Data Quality Monitoring Overview
- DLT Expectations
- DQX Library (Databricks Labs)
- Lakehouse Monitoring
- Anomaly Detection
- Data Profiling
- Custom Metrics